



CS683: Advanced Computer Architecture

Lecture 20: out-of-order, re-order, rename, all demystified 😊

<https://www.cse.iitb.ac.in/~biswa/courses/>

<https://www.cse.iitb.ac.in/~biswa/>

Exceptions and all

- Tomasulo (Last week's out of order with reservation station, out of order completion/commit):
 - In-order issue, out-of-order execution, **out-of-order completion**
- **Recall:** An interrupt or exception is precise if there is a single instruction for which:
 - All instructions before that have committed their state
 - No following instructions (including the interrupting instruction) have modified any state.
- **Need way to resynchronize execution with instruction stream (I.e. with issue-order)**

Re-order Buffer (ROB)

- Out-of-order commit: What about precise exceptions ?
- It would be great to have in-order commit with O3 execute.
- The process of an instruction leaving the ROB will now be called as **commit**.
- To preserve precise exceptions, a result is written into the register file only when the instruction commits (Ishan's R1 and R100 😊).
- Until then, the result is saved in a temporary register in the ROB.

Speculative Tomasulo with ROB

1. **Dispatch and Issue**—get instruction from Op Queue

If reservation station **and reorder buffer slot** free, issue instr & send operands & **reorder buffer no. for destination (“Issue”)**

2. **Execution**—operate on operands (EX)

When both operands ready then execute; if not ready, watch CDB for result; when both in reservation station, execute; checks RAW (sometimes called “issue”)

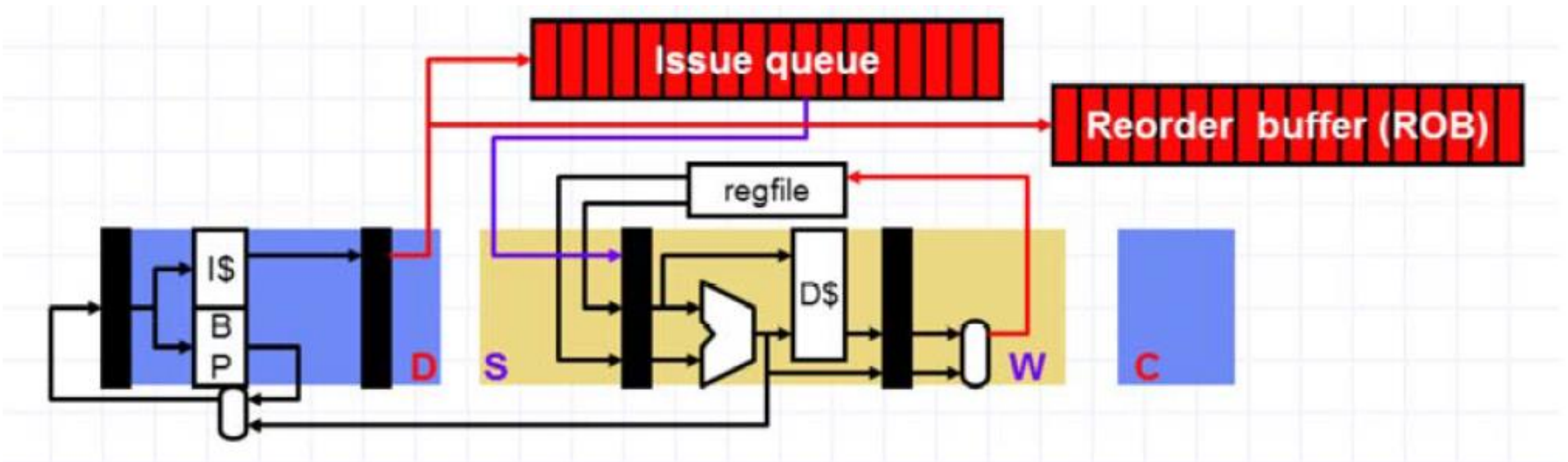
3. **Write result**—finish execution (WB)

Write on Common Data Bus to all awaiting FUs
& **reorder buffer**; mark reservation station available.

4. **Commit**—update register with reorder result

When instr. at head of reorder buffer & result present, update register with result (or store to memory) and remove instr from reorder buffer. Mispredicted branch flushes reorder buffer (sometimes called “graduation” or “retire”)

Issue Q + ROB



The subtle points

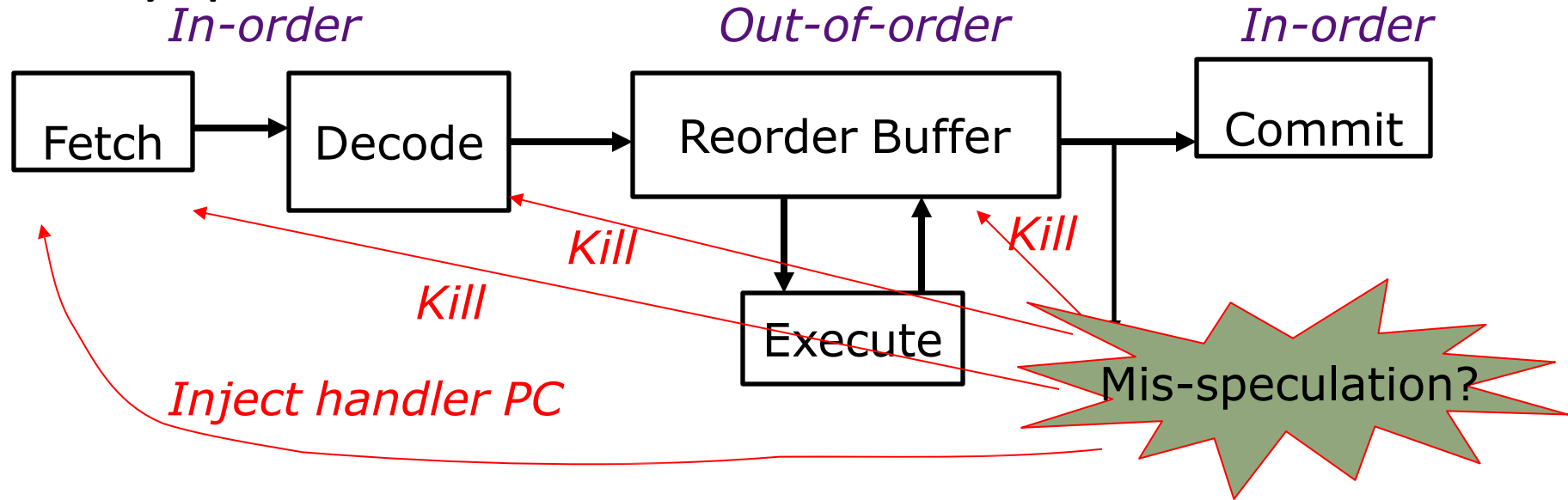
Issue Q : 10s of entries (64)

ROB: 100s of entries (352, 512, 750)

Issue Q: Only for not-issued instructions

Waking-up 10s of instructions versus 100s 😊

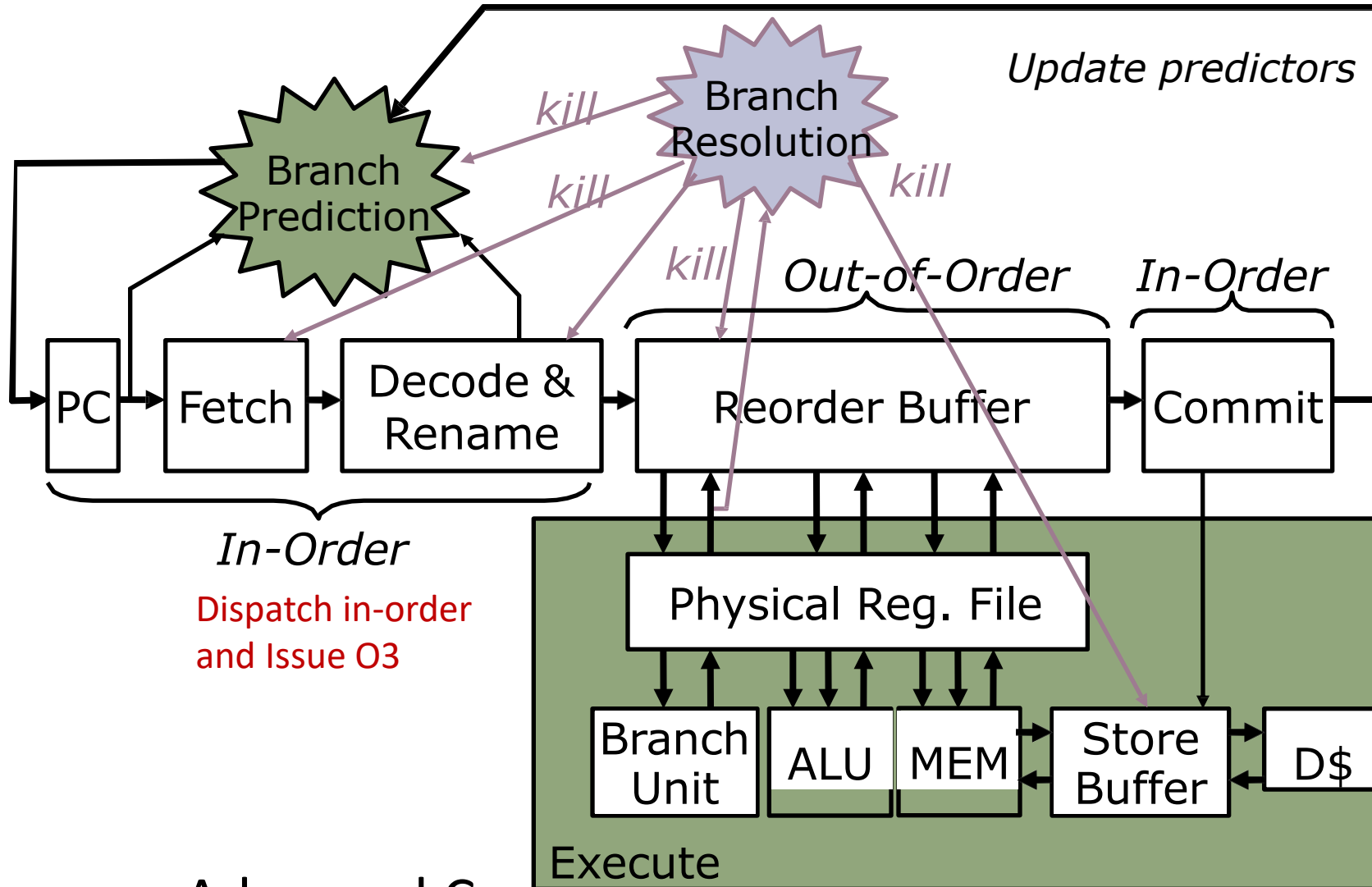
Recovery phase



- Instructions fetched and decoded into instruction reorder buffer in-order
- Execution is out-of-order ($\Rightarrow$ out-of-order completion)
- *Commit* (write-back to architectural state, i.e., regfile & memory) is in-order

*Temporary storage needed to hold results before commit
(shadow registers and store buffers)*

The complete pipeline



Transient/Speculative for CS773 !!

Transient instruction

- A speculative instruction that does not commit is a **transient** instruction

Speculative Execution-II

1. Proceed ahead despite unresolved dependencies using a prediction for an architectural or micro-architectural value



2. Maintain both old and new values on updates to architectural (and often micro-architectural) state

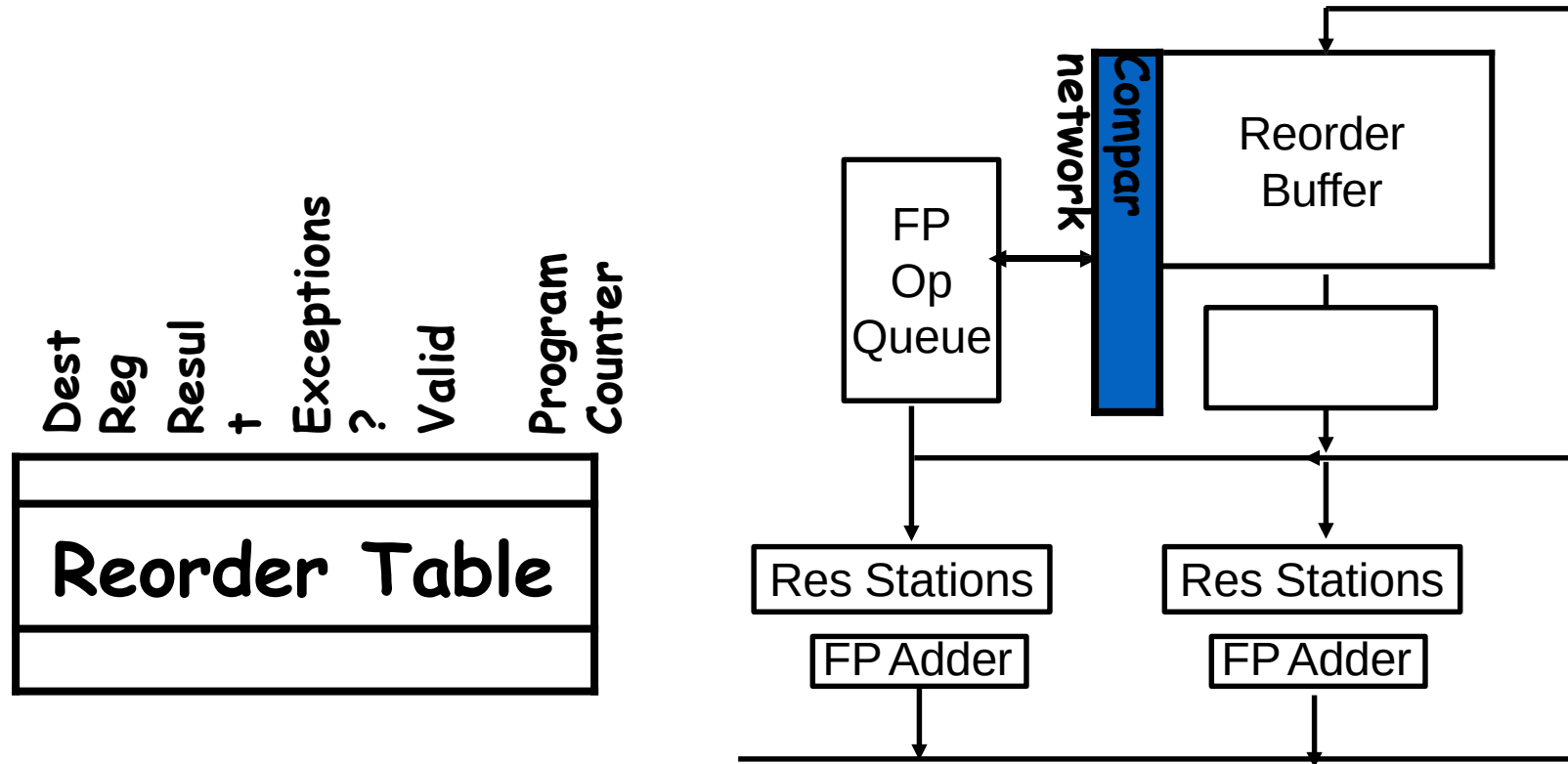
After speculation check

3. After sure that there was no mis-speculation and there will be no more uses of the old values, discard old values and just use new values

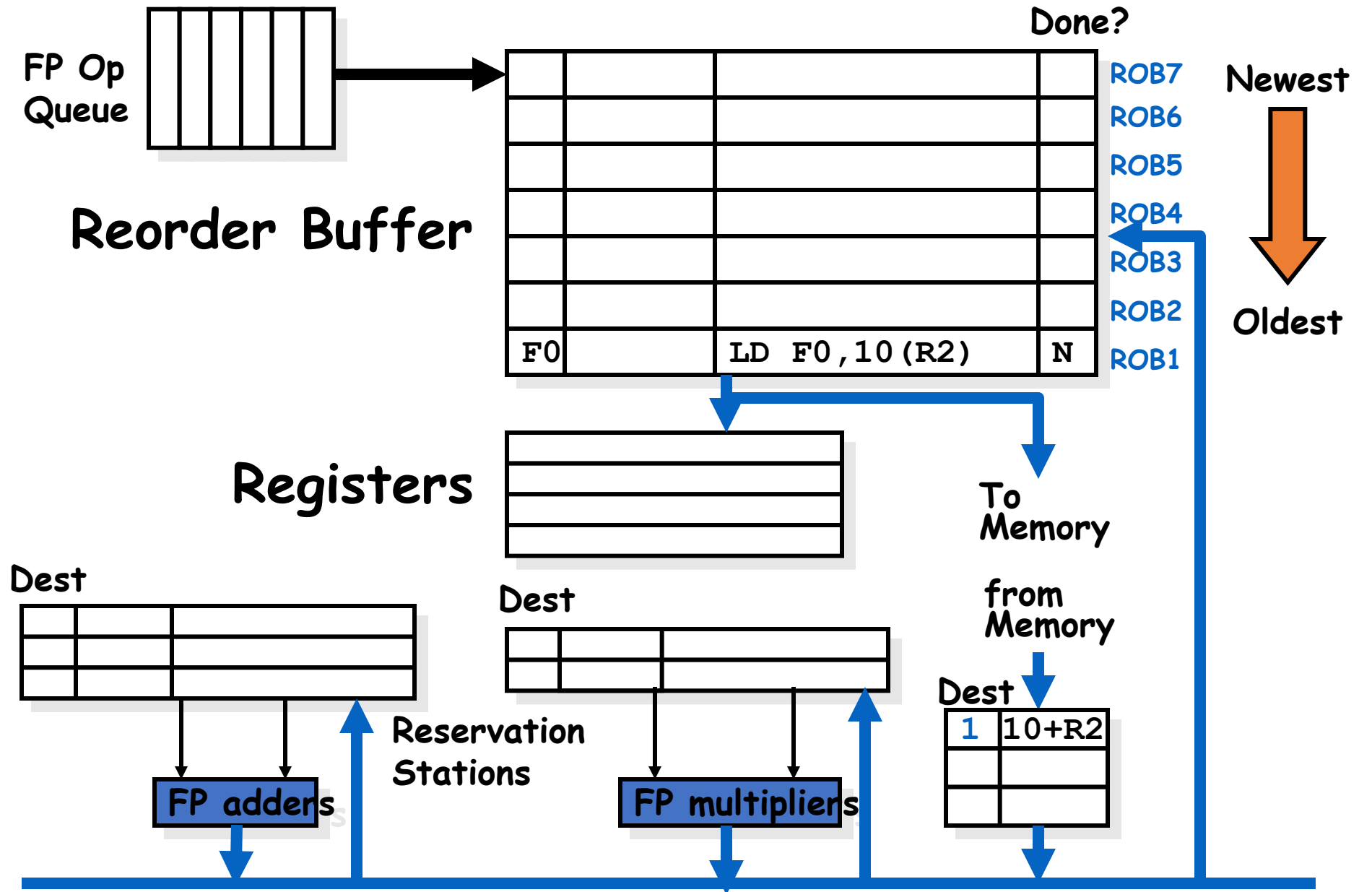
OR

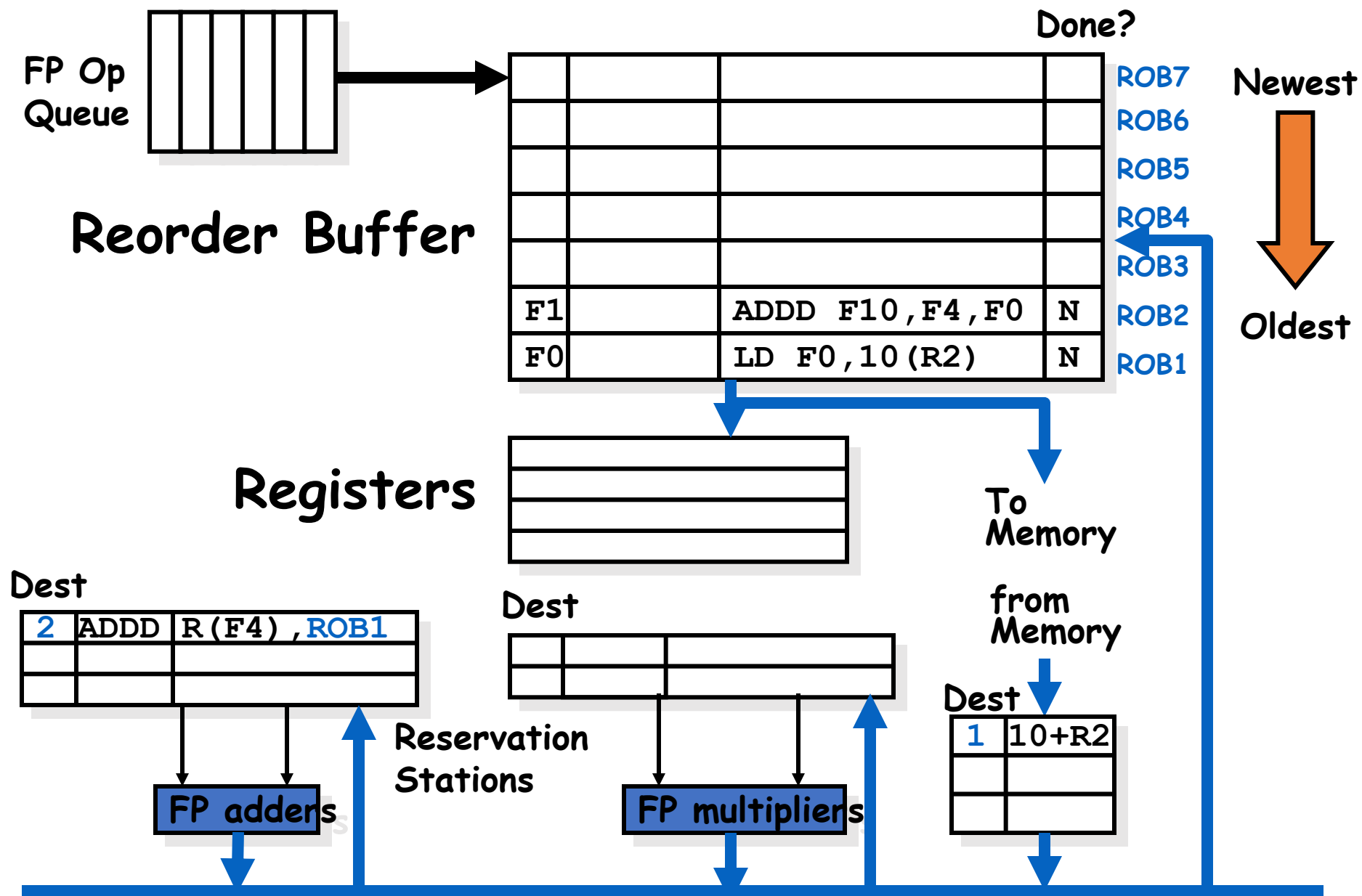
3. In event of mis-speculation, dispose of all new values, restore old values, and re-execute from point before mis-speculation

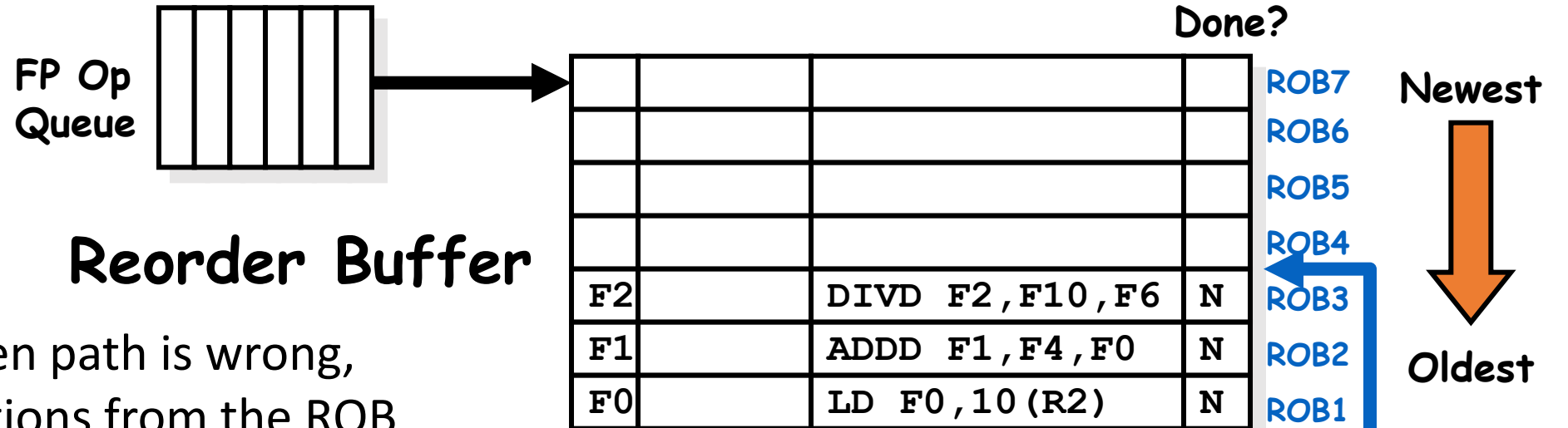
ROB



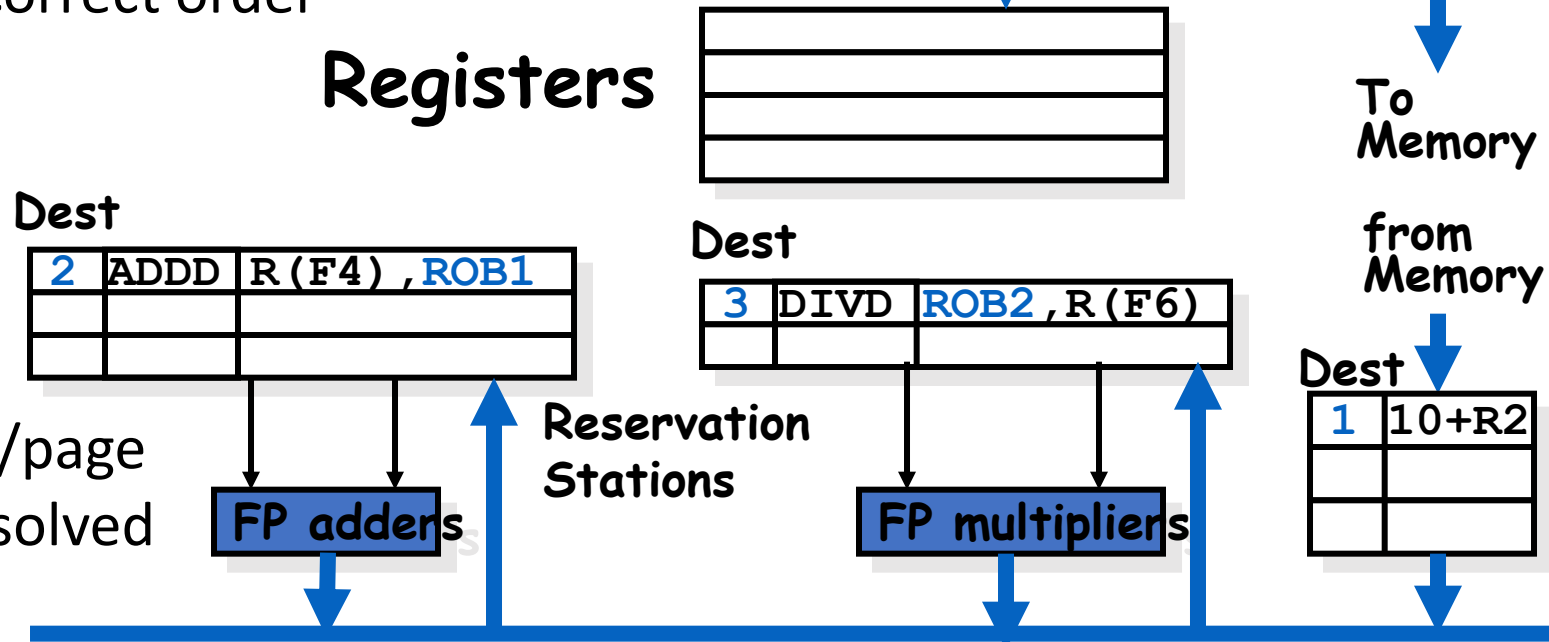
Please note that ROB stores the results too 😊 , When commit/retire, it updates registers and memory





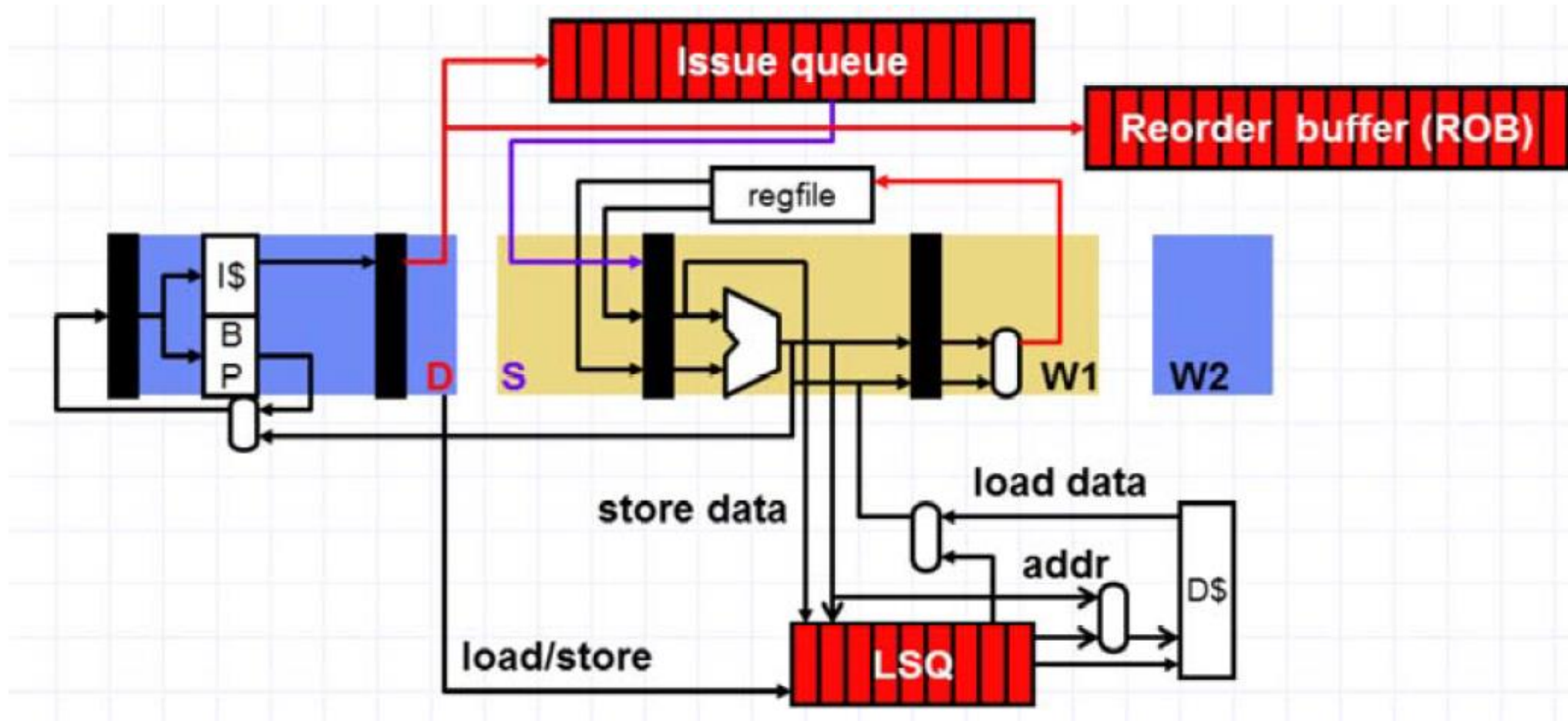


If the predicted taken path is wrong, flush out all instructions from the ROB and reissue in the correct order



Remember exception/page fault handling gets resolved when the instruction reaches the head of the ROB

LSQ (Load Store Q): For out-of-order requests into memory hierarchy



Why LSQ?

- ROB makes sure register writes are in order. What about stores or memory writes?
- LSQ:
 - i) Completed stores write to LSQ, in an O3.
 - ii) When store **retires**, head of LSQ written to L1D
 - iii) When load executes, access LSQ and L1D in parallel.
 - iv) Loads are O3. Else there is no point 😊

Continues.... LQ and SQ, On a load request

- Select the youngest store older than the load
- If hit in the SQ, forward data from SQ
- If miss, send it to TLB and L1D.

The tricky situation

`sd x1, (x2)`

`ld x3, (x4)`

When can we execute the load?

Approaches

- **In-order:** ☹️ Then what is the point of out-of-order ☹️ ☹️
- Can execute load before store, **if addresses known** and $x4 \neq x2$
- Each load address compared with addresses of all previous uncommitted stores, do not LOAD if STORE address is not known

Dependency violation continues

- LOAD executed before STORE ☹️ Fix it ☹️
- How?

Select **the oldest load that is younger than the store**

Use store position, Processor flushes and restarts.

Address Speculation

sd x1, (x2)

ld x3, (x4)

- **Guess** that $x4 \neq x2$
- Execute load before store address known
- Need to **hold all completed but uncommitted load/store addresses in program order**
- If subsequently find $x4 == x2$, squash load and all following instructions => Large penalty for inaccurate address speculation

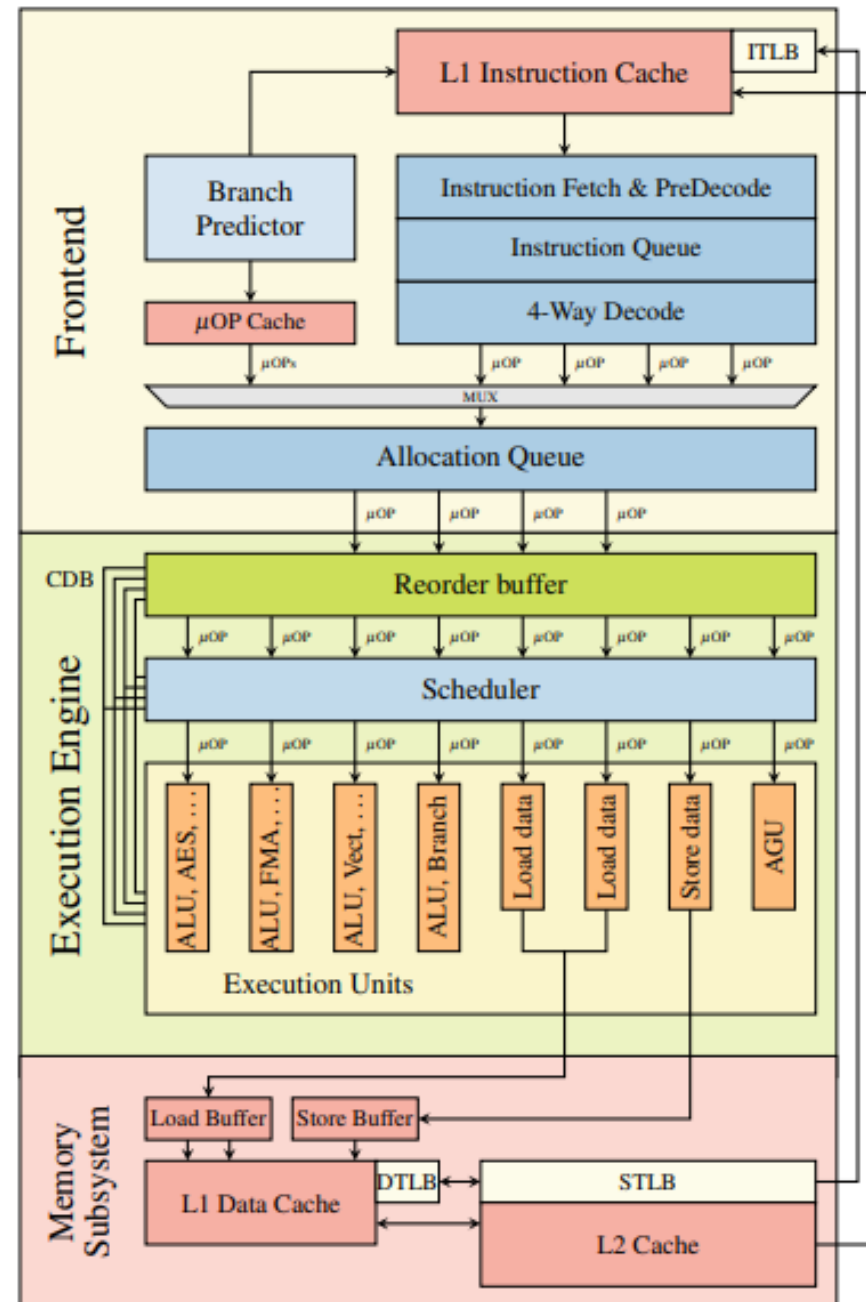
Better Approach? Predict, Load-store dependency predictor

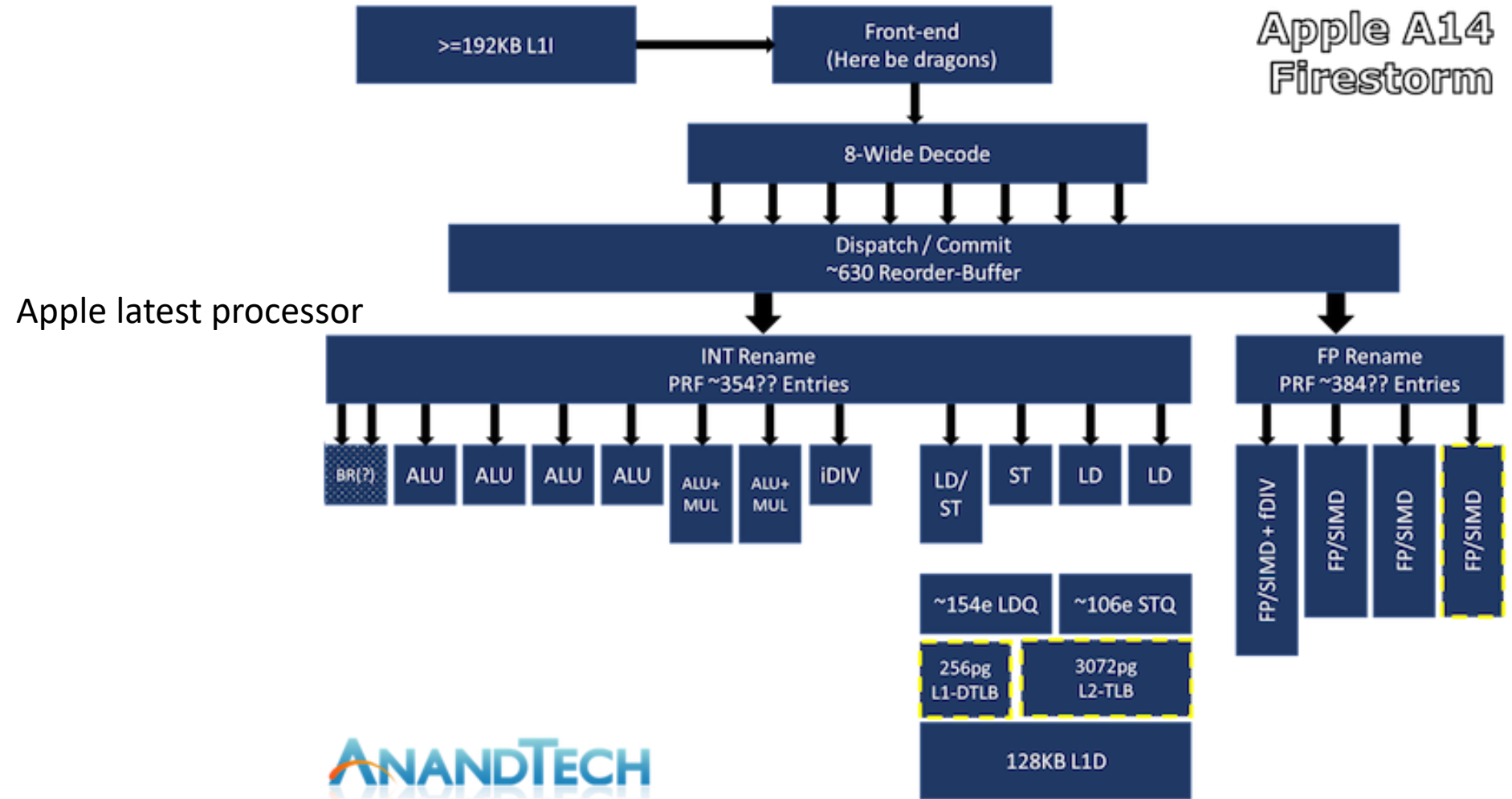
- Predict, which loads or load store pairs will cause violations.
- Loads **must execute in-order with respect to older stores.**
- Load execution requires knowledge of all older store addresses. If some older store address is not available yet, wait 😞

Do not Forget the Bigger Picture

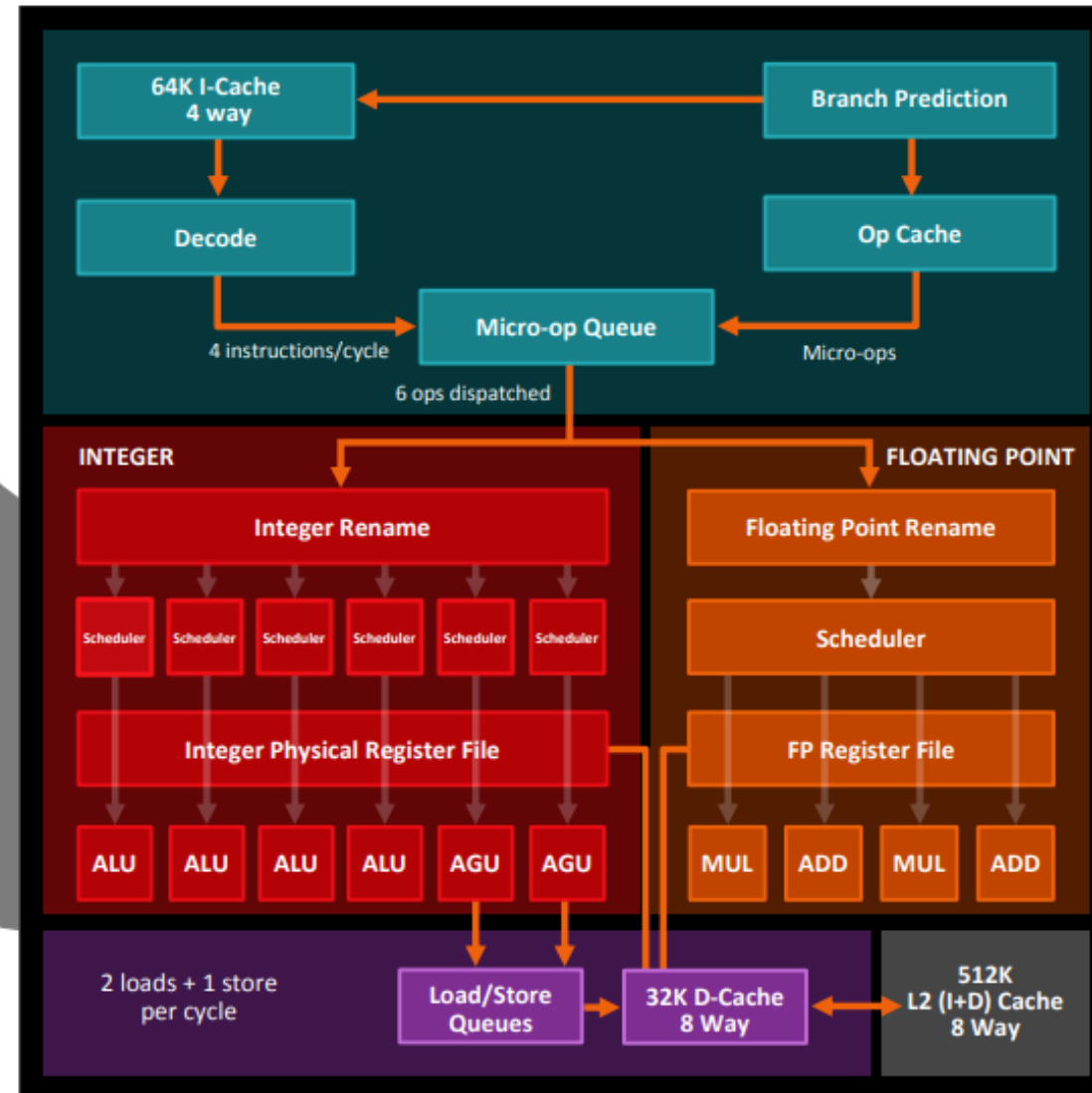
- Issue Q
- ROB
- RS
- LQ
- SQ

Intel latest processor





AMD latest processor





Devil is in the details

Connect All now

Memory systems, caches, TLBs, MSHRs, LSQs, O3 execution, ROB,

Can you guess: An L3 cache miss may not cause performance degradation, why? When? 😊

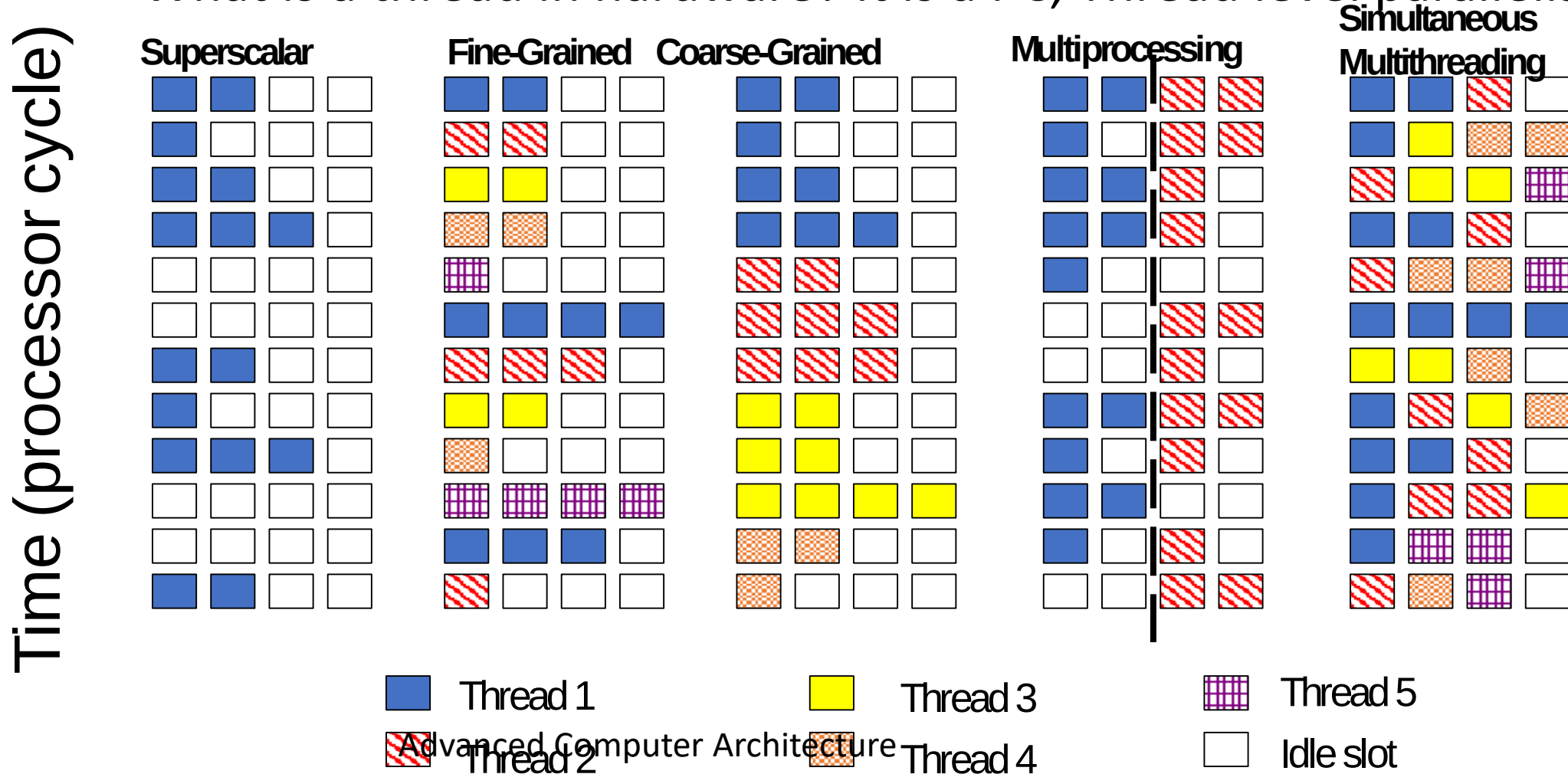
Let's connect all on Friday 😊

What next: Energy/Power of a modern processor

SMT processors: ILP to TLP (Thread-level)

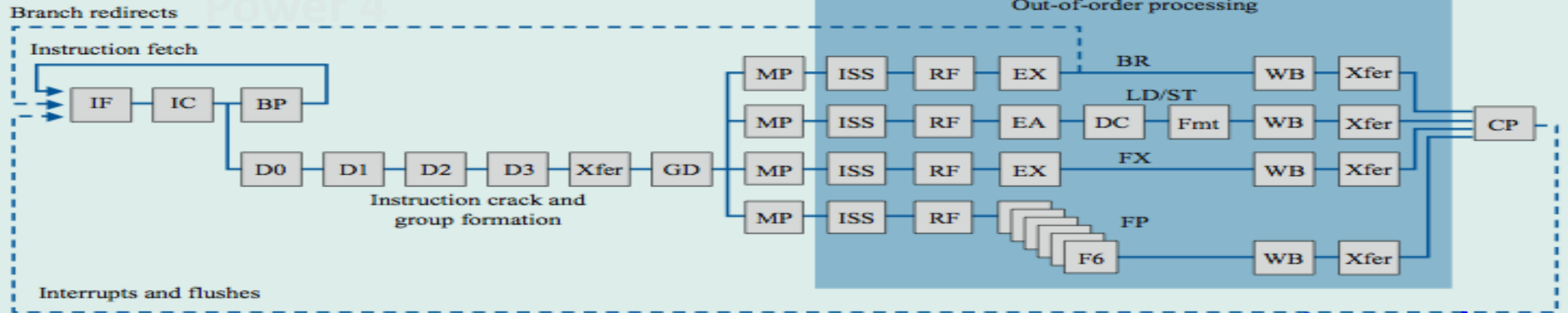
Single-thread to SMT (Yes IBM, no Apple)

- What is a thread in hardware? It is a PC, Thread level parallelism



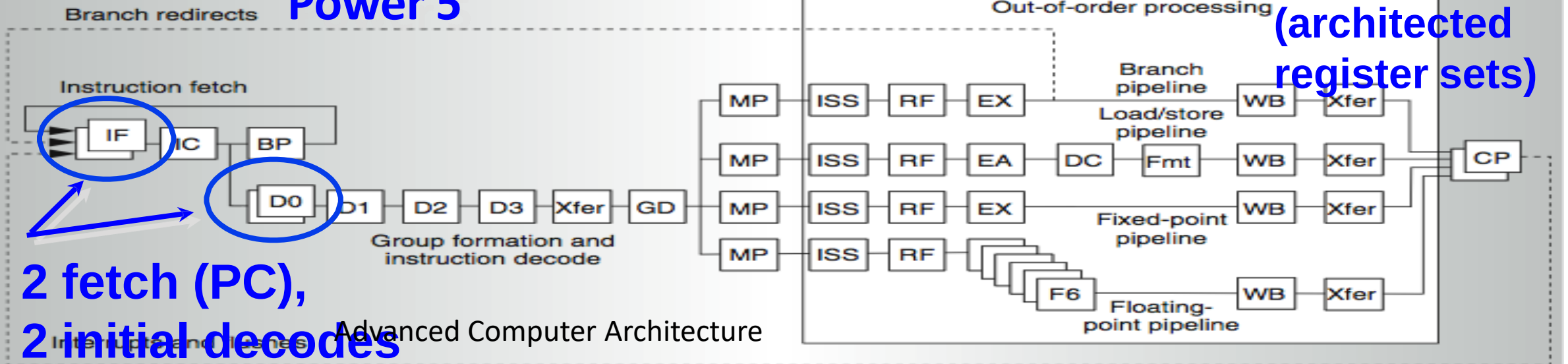
IBM Power Series

Power 4



2 commits
(architected
register sets)

Power 5



2 fetch (PC),
2 initial decodes